

MOMAI

O Sistema Operacional de IA para Seu Computador

Privado. Local. Inteligente.
Código Aberto. 100% Offline.

MomAI é um assistente virtual que vive no seu computador — não na nuvem. Ele vê sua tela, ouve sua voz, entende seu contexto e age por você. Tudo roda localmente: inferencia, memoria, automacao. Zero dados enviados para fora.

O Que e MomAI?

MomAI e um sistema cognitivo de proposito geral que transforma seu computador em um ambiente inteligente e autonomo. Diferente de assistentes tradicionais (Siri, Alexa, Google Assistant) que sao caixas-pretas na nuvem, a MomAI e:

- 100% local — todo processamento acontece na sua maquina, sem depender de internet
- Privada por construcao — seus dados nunca saem do seu computador
- Codigo aberto (MIT) — auditavel, modificavel, sem lock-in de fornecedor
- Extensivel — qualquer desenvolvedor pode criar modulos e skills
- Proativa — nao apenas reage, mas antecipa e age autonomamente
- Multi-modal — texto, voz, visao computacional e controle de dispositivos

Sumario

3 O Problema

4 A Solucao: Arquitetura Atual

5 Node Core: O Cerebro

6 Ecossistema de Skills e Extensoes

7 MomAI Events: A Nova Arquitetura

8 Ciclo Cognitivo em 6 Etapas

9 Hub, Edge e Auto-Modificacao

10 Request-Response + Event-Driven

11 Roteiro e Timeline

12 Por Que MomAI Vence

O Problema

Os assistentes de IA de hoje sao visitantes no seu computador.

Dados vao para nuvem

Cada conversa, cada comando, cada arquivo processado por assistentes como ChatGPT, Copilot ou Gemini e enviado para servidores remotos. Voce nao tem controle sobre o que e armazenado ou como e usado.

Custo recorrente

Modelos de assinatura (ChatGPT Plus, GitHub Copilot) cobram de \$20 a \$200 por mes. Empresas com centenas de funcionarios pagam fortunas por licencas que poderiam rodar localmente.

Dependente de internet

Sem conexao, sem assistente. Em viagens, areas remotas, ou durante quedas de rede, o computador volta a ser "burro".

Baixa integracao com o sistema

Assistentes na nuvem nao veem sua tela, nao controlam seus aplicativos, nao acessam seus arquivos locais. Eles sao caixas de texto sofisticadas, nao sistemas operacionais.

Latencia impraticavel para automacao

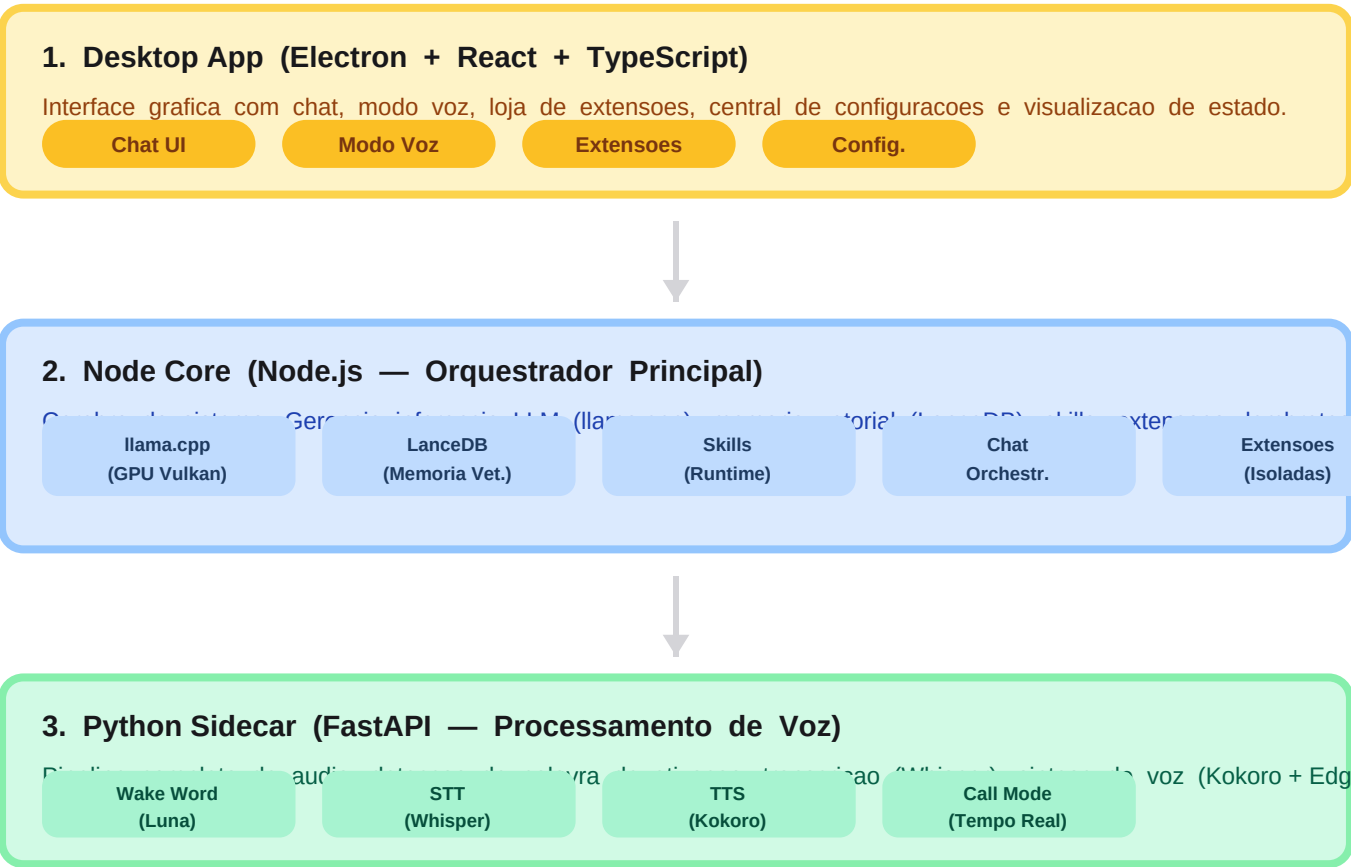
Um ciclo de ida-e-volta para a nuvem leva 500-2000ms. Para automacao domestica ou reacoes em tempo real, isso e inviavel.

Vendor lock-in

Cada assistente tem seu ecossistema fechado. Alexa nao fala com Google Home. Siri nao integra com nada fora da Apple. O usuario fica preso a um fornecedor.

A Solucao: Arquitetura Atual

A MomAI ja existe e funciona. Sua arquitetura atual e composta por tres processos independentes que se comunicam via HTTP, WebSocket e IPC. Cada um tem uma responsabilidade clara e pode ser substituido ou atualizado sem impactar os demais.



Node Core: O Cerebro em Detalhe

O Node Core é o componente central da MomAI. Ele orquestra toda a comunicação entre o usuário, o LLM e as skills. Diferente de sistemas tradicionais onde a lógica é fixa em código, aqui o LLM decide dinamicamente o que fazer.

Inferencia Local (llama.cpp)

Gerencia todo o ciclo de vida do llama-server: inicialização, seleção de GPU (Vulkan/CUDA/CPU), download automático de modelos GGUF do HuggingFace e fallback entre portas. Suporta 3 tiers de modelo: Lite (0.8B), Pro (2B) e Ultra (4B) — cada um com janela de contexto e capacidades diferentes.

Memoria Semantica (LanceDB)

Banco vetorial local para recuperação de contexto entre sessões. Quando o usuário faz uma pergunta, o sistema busca notas e memórias relevantes usando busca vetorial + lexical (fallback). O resultado é injetado no prompt do LLM como contexto de memória.

Orquestracao de Chat com Tools

O fluxo é: mensagem do usuário -> recupera memória -> descobre skills relevantes (top-5 por similaridade semântica) -> converte skills em tools no formato OpenAI -> envia para o LLM -> LLM decide entre responder ou chamar tool -> executa tool -> retorna resultado para LLM -> resposta final. Tudo via SSE streaming.

Skill Discovery Inteligente

Não precisa configurar nada. O sistema descobre automaticamente qual skill ativar baseado na intenção do usuário: busca semântica (LanceDB) nos nomes, descrições e tools das skills. Fallback lexical por palavras-chave. Top 5 viram ferramentas disponíveis para o LLM.

Streaming SSE e WebSocket

Toda resposta do LLM é transmitida em tempo real via Server-Sent Events para o frontend. O WebSocket bidirecional transmite eventos de voz (início/fim de TTS), progresso de inicialização, uso de recursos, e traces de observabilidade.

Persistencia Local

Tudo é salvo em arquivos JSON locais: configurações, threads de conversa, lembretes, preferências. Nenhum banco de dados externo. Nenhuma telemetria. Os dados são seus.

Ecosystem of Skills and Extensions

A MomAI is extensible by design. Skills and extensions are modules that add capabilities to the system. The LLM never calls hardware directly — everything passes through intermediary modules.



Principle: The LLM never communicates directly with hardware. Every interaction passes through intermediary modules. If a module changes brand or protocol, only the module needs to be updated — the LLM continues calling the same tool.

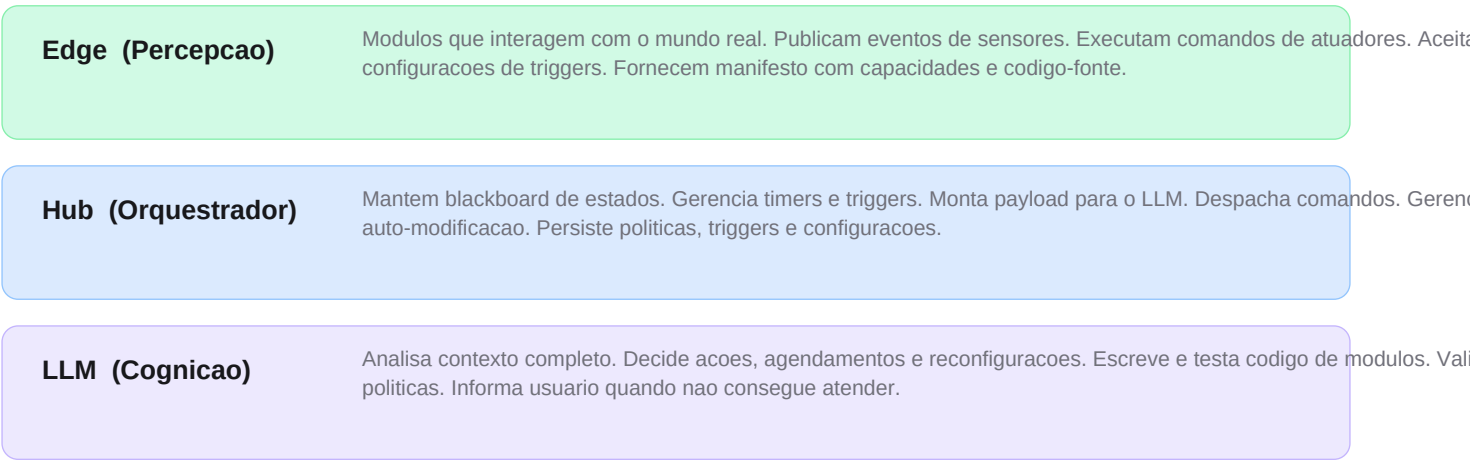
MomAI Events: A Nova Arquitetura

Enquanto a arquitetura atual resolve o problema de "assistente conversacional", o MomAI Events e a evolucao para um sistema cognitivo autonomo e orientado a eventos. Inspirado pelo Jarvis (Tony Stark) e pela Abelha Rainha (Resident Evil), ele transforma a MomAI de reativa para proativa.

Tres Pilares Fundamentais

- Zero Hardcode**
O nucleo do sistema nao sabe o que esta conectado. Nenhum dispositivo, protocolo ou rotina esta fixado no codigo. Tudo e descoberto dinamicamente atraves de manifestos.
- Event-Driven**
Tudo no sistema gira em torno de eventos. Sensores publicam mudancas. Timers disparam. O LLM so acorda quando algo relevante acontece. Nao ha polling ou checagem constante.
- Auto-Modificacao**
O LLM pode ler, modificar e estender o codigo dos modulos intermediarios para atender requisitos que os modulos originais nao cobrem. Tudo testado em sandbox antes de aplicar.

Arquitetura em 3 Camadas



Ciclo Cognitivo em 6 Etapas

O ciclo cognitivo é o processo central do MomAI Events. Toda vez que o LLM é invocado para decidir, ele segue estas 6 etapas:

1. Algo Acontece

Um evento é publicado no barramento: sensor detectou mudança, timer estourou, comando de voz do usuário, ou um trigger composto foi ativado.

2. Hub Monta o Contexto

O Orquestrador interrompe timers pendentes e monta o payload: estado atual de todos os sensores, manifestos dos módulos, políticas ativas do usuário, registro de triggers configurados em ciclos anteriores.

3. LLM Processa

O modelo analisa o cenário completo, correlaciona o evento com as políticas ativas, e decide: ações imediatas, auto-agendamento, ou reconfiguração de módulos.

4. LLM Responde

Resposta estruturada em três blocos: (A) ações para módulos, (B) auto-agendamento com tempo de espera e gatilhos de interrupção, (C) reconfigurações nos módulos (incluindo edição de código).

5. Hub Executa

Despacha comandos para os módulos corretos. Se houver reconfigurações, gerencia o ciclo de edição -> teste em sandbox -> aplicação ou rollback.

6. Sistema Espera

Timer criado com o tempo definido pelo LLM. Se um gatilho de interrupção acontecer antes, o ciclo recomeça. Se o timer estourar sem interrupção, o sistema acorda sozinho. Consumo mínimo durante espera.

E enquanto espera? Consumo mínimo. O LLM só carrega quando invocado.

Entre ciclos cognitivos, o sistema consome recursos mínimos. Sem polling. Sem checagem constante. Só eventos.

Hub, Edge e Auto-Modificacao

Detalhando os componentes mais inovadores da nova arquitetura.

Hub (Orquestrador)

- Blackboard: snapshot do estado atual de todos os sensores e modulos conectados
- Fila de timers: gerencia triggers temporais definidos pelo LLM e pelo usuario
- Payload montagem: combina estado + manifestos + politicas + triggers para o LLM
- Despacho: envia comandos para os modulos corretos no formato certo
- Persistencia: politicas, triggers e configuracoes sobrevivem a reinicializacoes

Edge (Modulos de Percepcao)

- Cada modulo publica um manifesto: nome, categoria (sensor/actuator/hybrid),
- capacidades, esquema de dados, comandos tipados, dependencias, estrutura de codigo
- Exemplos: Camera (visao), Microfone (audio), Iluminacao, Climatizacao, Travas, TV, Geladeira
- Modulos podem ser escritos em qualquer linguagem (Node.js, Python, etc.)

Auto-Modificacao de Modulos

- Fase 1 - Analise: LLM le o manifesto e codigo-fonte do modulo
- Fase 2 - Planejamento: define alteracoes, dependencias, novas tools
- Fase 3 - Sandbox: cria copia isolada, implementa, instala dependencias
- Fase 4 - Validacao: testes para garantir que nao ha regressoes
- Fase 5 - Aplicacao: se testes passam, novo modulo vai para producao
- Fase 6 - Rollback: se falha, tenta corrigir. Apos limite, aborta e avisa usuario

Request-Response + Event-Driven

Uma das decisoes de arquitetura mais importantes: nao precisamos escolher entre os dois modelos. O Hub opera em modo hibrido, atendendo tanto requisicoes diretas do usuario quanto eventos autonomos.

Duas Portas de Entrada, Um Nucleo

Chat Path

- Usuario digita ou fala
- Montagem de contexto
- LLM processa e responde
- Skills chamadas se necessario
- Resposta em streaming SSE
 - TTS automatico ativado
 - Ciclo simples e rapido
- Mesmo LLM, mesmas skills
- Experiencia de chat familiar

Event Path

- Sensor publica evento
- Trigger avalia relevancia
- Se match: monta payload
- LLM decide o que fazer
- Acoes + auto-agendamento
- Timer criado com gatilhos
- Pode interromper timers anteriores
- Ciclo cognitivo completo
- Operacao autonomo continua

Compartilhado: LLM | Skills | Modulos | Politicas | Memoria | Persistencia

Ambos os caminhos resolvem para a mesma operacao: montar contexto -> LLM decidir -> agir. A diferenca e quem inicia o ciclo - o usuario ou um evento. O Node Core atual ja tem 90% da infraestrutura necessaria para o Hub. Faltam: barramento de eventos, sistema de triggers, e politicas como dados persistentes.

Roteiro e Timeline

O que ja foi construido e o que vem pela frente.

HOJE	BASE ATUAL Desktop funcional com chat, voz, skills, extensoes, memoria semantica, LLM local com aceleracao GPU. Tudo funcionando e testado.
Q3 2026	Q3 2026 MomAI Events - Implementacao do Hub com barramento de eventos, sistema de triggers simples e compostos, politicas ativas persistentes, e ciclo cognitivo basico.
Q4 2026	Q4 2026 Computer Use - Visao computacional para entender a tela e controlar aplicativos visualmente. O computador passa a "ver" o que o usuario ve.
Q1 2027	Q1 2027 Auto-Modificacao - Sandbox para edicao de modulos pelo LLM. Versionamento de modificacoes, rollback automatico, instalacao segura de dependencias.
Q2 2027	Q2 2027 Marketplace de Extensoes - Loja para criadores terceiros publicarem modulos. Efeitos de rede: mais modulos atraem mais usuarios, mais usuarios atraem mais criadores.
H2 2027	H2 2027 Mobile + Enterprise - App leve para iOS/Android com sincronizacao. Modo empresa com gerenciamento centralizado, auditoria e politicas de equipe.

Por Que MomAI Vence

- **Privacidade como Direito, Não Feature**

Enquanto concorrentes Monetizam seus dados, a MomAI os protege. Em um mundo onde toda empresa quer seus dados, a MomAI é a única que os recusa. Nenhuma telemetria. Nenhum dado sai da sua máquina.

- **Mercado em Movimento**

Apple Intelligence, Microsoft Copilot, Google Gemini — todas as big techs estão apostando em IA local. Mas são jardins murados. A MomAI é a alternativa aberta. O mercado de IA local deve crescer de \$15B (2025) para mais de \$100B em 2030.

- **Executando Hoje**

Não é um slide. É um produto funcional. Código-fonte aberto no GitHub. Download e uso imediato. Não estamos esperando um prototipo — estamos evoluindo uma base real.

- **Custo Zero de Operação**

Zero custo de nuvem. Zero assinatura. O usuário usa o próprio hardware. Para empresas, isso significa eliminar custos recorrentes de licenças de IA que já chegam a \$200/usuário/mes.

- **Open Source como Moat**

Código aberto não é doação — é estratégia. Quanto mais pessoas usam, mais extensões surgem. Mais extensões atraem mais usuários. Efeitos de rede que concorrentes fechados não conseguem replicar. A comunidade vira o motor de crescimento.

- **Arquitetura a Prova de Futuro**

O design em 3 camadas com módulos intermediários significa que novos hardwares, novos modelos de IA e novos protocolos se integram sem reescrever o núcleo. Auto-modificação permite que o próprio sistema se adapte a necessidades não previstas.

[**github.com/WesleyQDev**](https://github.com/WesleyQDev)

MomAI | Open Source | MIT License